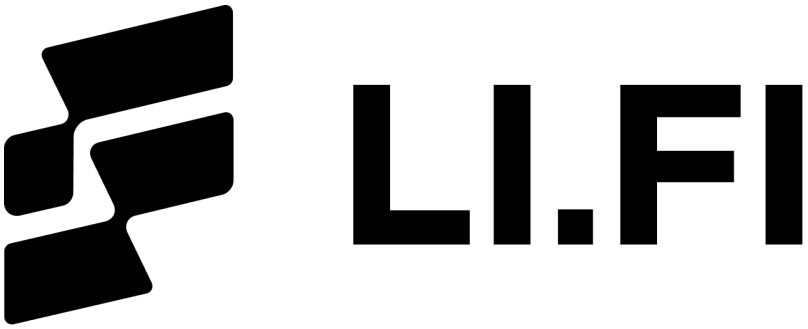

LI.FI Security Review

Tron Canonical USDT Migration - Part 1

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

May 12, 2026

Contents

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/GenericSwapFacetV3.sol(v2.0.0)
- src/Helpers/WithdrawablePeriphery.sol(v2.0.0)
- src/Periphery/LiFiDEXAggregator.sol(v1.13.0)
- src/Periphery/ReceiverAcrossV3.sol(v1.2.0)
- src/Periphery/ReceiverChainflip.sol(v1.1.0)
- src/Periphery/ReceiverStargateV2.sol(v1.2.0)
- src/Periphery/TokenWrapper.sol(v1.3.0)

4 Risk classification

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

High leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.

Medium global losses <10% or losses to only a subset of users, but still unacceptable.

Low losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

High almost certain to happen, easy to perform, or not easy but highly incentivized

Medium only conditionally possible or incentivized, but still relatively likely

Low requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Critical Must fix as soon as possible (if already deployed)

High Must fix (before deployment if not already deployed)

Medium Should fix

Low Could fix

5 Executive Summary

Over the course of 10.2 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 3 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	142d9d8
Audit Timeline	May 8, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Gas Optimizations	0
Informational	1
Total Issues	**3**

6 Findings

6.1 Low Risk

6.1.1 The `_swapAndCompleteBridgeTokens()` function in `ReceiverChainflip` leaves executor allowance non-zero on the success path

Context: ReceiverChainflip.sol#L135

Description: `ReceiverChainflip` short-circuits with return; on the executor try-success branch, bypassing the `actualAssetId.safeApprove(address(executor), 0)` reset. If the executor consumes less than the full approved amount in any future change (today it consumes the full bridged amount, but executor evolution is not pinned by this contract), residual allowance persists across calls. This pattern is asymmetric with `ReceiverAcrossV3`, which always resets regardless of try/catch outcome.

The issue pre-existed in v1.0.1; this PR did not introduce it. It is called out here because the file is being touched and the minor version is bumping to v1.1.0, so the fix can be made in the same series with no incremental review cost.

Recommendation: Restructure `ReceiverChainflip._swapAndCompleteBridgeTokens` so the approval reset runs on both branches:

```
if (!isNative) {
      actualAssetId.safeApproveWithRetry(address(executor), amount);
      try executor.swapAndCompleteBridgeTokens(_transactionId, _swapData, actualAssetId, receiver) {
      ↪ +          // success  fall through to reset
-          return
      } catch {
          LibAsset.transferERC20(actualAssetId, receiver, amount);
          emit LiFiTransferRecovered(_transactionId, actualAssetId, receiver, amount, block.timestamp);
      }
      actualAssetId.safeApprove(address(executor), 0);
  }
```

LI.FI: Fixed in 95647cf. We had previously identified the same issue in our bug bounty program and, at the time, classified it as an inconsistency rather than a security vulnerability, so no action was taken.

Since this PR already modifies the relevant file, it's a good opportunity to address the issue and improve consistency. We've removed the early return from the success branch, ensuring that safeApprove(address(executor), 0) is always executed. This aligns the behavior with the pattern used across the other receiver contracts.

Researcher: Verified fix.

6.1.2 Switching catch-block transfer to LibAsset.transferERC20 makes `receiver == address(0)` a hard revert, suppressing `LiFiTransferRecovered` emission

Context: ReceiverStargateV2.sol#L216, ReceiverStargateV2.sol#L197, ReceiverChainflip.sol#L137, ReceiverAcrossV3.sol#L106

Description: The current change replaces the catch-block raw `safeTransfer()` calls in the three receivers with `LibAsset.transferERC20`.

The two functions are not behaviorally equivalent: `LibAsset.transferERC20` reverts unconditionally with `InvalidReceiver` when recipient == address(0), while the prior `safeTransfer()` deferred zero-address handling to the underlying token. The PR therefore introduces a new revert path: when a bridge message decodes `receiver = address(0)`, the catch block now reverts before `LiFiTransferRecovered` is emitted.

n all three receivers (ReceiverAcrossV3, ReceiverChainflip, ReceiverStargateV2), this revert propagates up through the bridge-protocol entry point (handleV3AcrossMessage / cfReceive / lzCompose) and the entire bridge attempt fails.

The Across SpokePool fill, Chainflip Vault call, or LayerZero Executor delivery reverts as a single unit. No tokens are credited to the receiver contract on the destination side; the user's source-chain funds remain locked in the bridge contracts (and can be recovered if there's any bridge specific refund mechanism).

The behavioral delta from this change is therefore narrow:

- For ERC20s that reject transfer(address(0), amount) (most production tokens, including USDC, USDT, DAI): no observable change.
- For ERC20s that treat transfer(address(0), amount) as a burn (a small minority): pre-PR, the user's tokens would be silently burned and `LiFiTransferRecovered` would be emitted with `receiver = address(0)`, leaving the user with no recourse on either chain. Post change, the bridge attempt fails entirely and the source-chain refund path engages. This is a strict improvement.

The `LiFiTransferRecovered` event is also no longer emitted in the zero-receiver case but since the bridge fails atomically, off-chain monitoring observing only successful bridge completions is unaffected, and observers tracking failed-fill / failed-compose events from the bridge protocols themselves still see the failure.

Recommendation: No code change required. The new behavior is uniformly correct across all bridges and improves outcomes for the burn-on-zero token edge case. Optionally document the change in the receivers' NatSpec so integrators understand the risk of locking funds if the bridge offers no recovery mechanism.

LI.FI: Agreed. no code changes are required; the new behavior is strictly safer. For the niche class of burn-on-zero tokens, the destination handler now reverts instead of silently burning funds. We've added concise NatSpec comments to each affected catch path documenting the InvalidReceiver behavior. Any post-revert recovery flow remains bridge-specific.

Researcher: Acknowledged.

6.2 Informational

6.2.1 Out-of-scope facets and LidoWrapper bypass LibAsset; the fork's single-point bypass does not reach them

Context: Global

Description: Outside that scope, the following call sites bypass `LibAsset` and would not benefit from any future `LibAsset`-level Tron USDT bypass handling.

File	Lines	Mechanism
`src/Facets/HopFacetPacked.sol`	259, 312, 592, 634	Solmate `ERC20.safeTransferFrom`
`src/Facets/AcrossFacetPacked.sol`	136, 182	Solmate `ERC20.safeTransferFrom`
`src/Facets/CBridgeFacetPacked.sol`	146, 184	Solmate `ERC20.safeTransferFrom`
`src/Facets/AcrossFacetPackedV3.sol`	164, 198	Solmate `ERC20.safeTransferFrom`
`src/Periphery/LidoWrapper.sol`	85, 98	Raw `IERC20.transfer` / `transferFrom` with no safe wrapper

The risk is forward-looking: if the team ever extends Tron coverage to Hop / Across / cBridge corridors, these facets will silently reintroduce the bug, and the LibAsset bypass alone will not catch it.

Recommendation: Add a docs/ page documenting the Tron deployment surface restriction. Explicitly list HopFacetPacked, AcrossFacetPacked, CBridgeFacetPacked, AcrossFacetPackedV3, and LidoWrapper as not Tron-safe in their current form.

For LidoWrapper, the raw IERC20.transfer pattern is a separate code-quality issue independent of Tron — replace with LibAsset.transferERC20 / LibAsset.transferFromERC20 to bring it in line with the rest of the periphery.

Alternatively, extend .solhint.json (or add a CI grep) that fails the build if any file in src/ (excluding src/Libraries/LibAsset.sol) contains .safeTransfer\b or .safeTransferFrom\b matched against an ERC20 receiver.

Allow safeTransferETH. This makes the architectural invariant ("all ERC20 value-flow goes through LibAsset") an enforced lint rule, not a convention.

LI.FI: Breaking down your three sub-points into what was addressed in this PR versus what remains intentionally unchanged:

- Packed facets (HopFacetPacked, AcrossFacetPacked, CBridgeFacetPacked, AcrossFacetPackedV3): these contracts are intentionally optimized for minimal gas overhead, and migrating their token handling to LibAsset would work against that design objective. They are also not deployed on Tron, and there are currently no plans to extend Tron support to these routes.
- LidoWrapper: agreed that this is primarily a code hygiene issue rather than a Tron-specific concern. Although in Lido both stETH and wstETH correctly return boolean values, the contract should still follow the same transfer safety patterns used throughout the codebase. This has been cleaned up as part of the current PR in the LidoWrapper v1.0.0 $\rightarrow$ v1.1.0 update: replaced raw IERC20.transfer / transferFrom calls with LibAsset.transferERC20 / LibAsset.transferFromERC20 migrated constructor approvals to Solady's safeApproveWithRetry
- All 12 existing tests continue to pass, with only two revert expectation selectors updated to match Solady's TransferFromFailed. Fixed in: debf4e8
- solhint / CI grep enforcement: nice idea, added ticket in our linear

Researcher: Verified fix and acknowledged all other points.